

Resit Exam Introduction to Databases

26 August 2019

- The final exam contains 5 problems and is scheduled for 4 hours.
- Please verify if your question booklet consists of 5 pages (plus 7 pages with supplementary lecture slides) that are printed legibly, else contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. For questions marked with an asterisk (*), *supplementary lecture slides* are provided at the end of this booklet.
- No electronic devices (such as notebooks, smartphones or smartwatches) are allowed.
- Usage of course material is not allowed.
- *Insufficiently detailed* answers are void (e.g.: you can't just answer "yes" or "no").
- Only the answer booklet will be collected, please do not use the question booklet for answering the questions.
- Take pride in your work: answer clearly and hand in a readable exam copy!
- For Problems 1.2, 1.3 and 2.1 you can complete the assignment using the available computers, which include postgres and DataGrip. Upload your solution to the SQL questions to blackboard (18-19: Introduction to databases > Opdrachten > Examen 26-08-2019 Examen). All other questions must be answered on paper.

Problem 1: Entity Relational Model and Relational Design (5 points)

Your task is to model a ticket booking platform. Carefully read the following requirement analysis. Remark that questions 1.2, and 1.3 can be created on the class computer and uploaded using blackboard.

A ticket booking platform is a marketplace where customers can buy tickets for events, such as concerts, shows and sport events. Different organizers can create events. An organizer logs on to the system, with his or her account, and can create one or more events. Each event has a major (e.g. "concert") and minor category (e.g. "pop-rock") and associated artist name (e.g. "Angèle"). An event is uniquely identified by the combination of the artist name, date, and venue. It is possible that an event is repeated at the same venue (e.g. "Ancienne Belgique" in Brussel) on different dates, or repeated at different venues on different dates, for example, when an artist goes on a tour. For each event there is a number of available tickets. Different types of tickets are possible, e.g. VIP seats are more expensive than standing seats. Remark that, for each type of ticket the price is set by the organizer of the event. For some events there's an early registration promotion, that is, if tickets are purchased before a certain date there's a discount of x%. Each ticket has a unique code, a seat number and a ticket type. A customer has a first and last name, birth date and unique e-mail address. Each customer can buy one or more available tickets for an event. When a ticket is bought, the date of purchase and price is registered by the system. Each customer can buy at most 10 tickets for a single event.

- ✓ 1. Create an Entity-Relationship diagram that models as faithfully as possible the above description. Clearly indicate entities, relationships, primary keys, cardinalities and participation constraints.
- ✓ 2. (*) Create the necessary create table statements to implement this ER-diagram as a relational database. Define all necessary schema-level constraints, such as primary keys, foreign keys and domain constraints. Active database instructions such as "On delete/update cascade/..." are not required.
3. Create a check to assert that tickets are bought at the correct price. The correct price is determined by the organizer of the event for each type of ticket. Also verify that the discount rate is applied if applicable.

Problem 2: SQL and Relational Algebra (5 points)

The Northwind database contains the sales data for a fictitious company called Northwind Traders, which imports and exports specialty foods from around the world. The Northwind schema is shown in figure 1. Download northwind.sql from blackboard (18-19: Introduction to databases > Opdrachten > Examen 26-08-2019) and run this script for creating tables and populating the database.

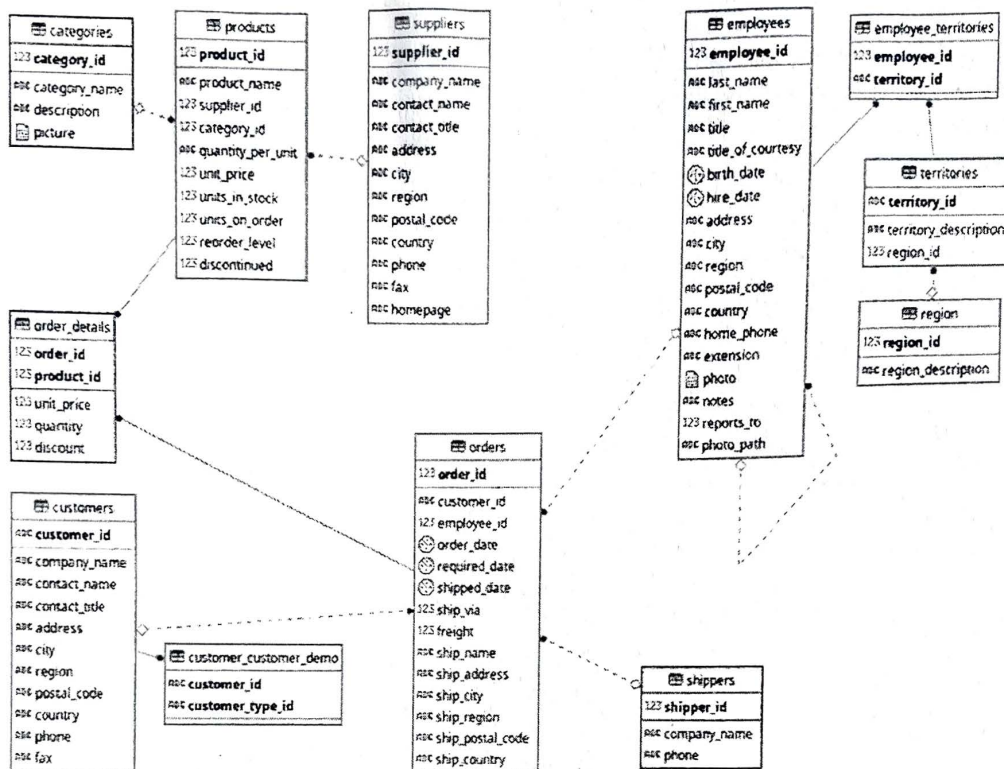


Figure 1. Northwind schema. In this notation, database relations are denoted by a rectangle. Key attributes are listed first in bold, followed by the other attributes. Foreign key dependencies are indicated with lines. The full dot indicates the table in which the foreign key is included that refers to the other table. Usually the name of the foreign key corresponds to the name of the primary key in the table referenced. One exception is the employees table where reports_to is a foreign key referencing back to the employees table. A full line indicates a defining relation; that is: the foreign key is part of the primary key (usually associated with a many-to-many relation in the ER model that became a database table, or a weak entity). A diamond means optionality of a relation; that is: the foreign key may be NULL.

1. Write the following queries in SQL:

- Find the first and last name of all employees that have processed orders from more than 100 customers.
- Find all pairs of product names that have been shipped by the same shipper from 'Belgium'.
- Find all pairs of product names that have never been ordered together. Ignore products that have been bought less than 40 times.
- List the company names of customers together with the total sum of quantity of products shipped from 'Germany'.

2. (*) Write the following queries in Relational Algebra:

- ↓ a. For each employee from the 'UK', list the name of the employee and the name of the employee to which he or she reports.
- b. Find customer and employee names living in the same country, where the customer has served the employee in purchasing an order.
- c. Find suppliers that do not produce all categories of products.

Problem 3: Database design (5 points)

Consider the following database relation that stores data about employees. Every employee has a unique identifier. Furthermore, for every employee his or her name, gender, address, bank account and salary are stored. Every employee works for exactly one department and is working on one or more projects. Each project has a unique project name and a project manager, who is also an employee. This data is stored into one relation. The following example shows how it is stored:

id	name	gender	address	department	account	salary	project	manager ID
1	Bert	M	Sesame St. 1	IT	010234	2200	Alpha	1
1	Bert	M	Sesame St. 1	IT	010234	2200	Beta	4
2	Ernie	M	Sesame St. 1	Accounting	010345	2400	Delta	3
3	Ruth	F	NE Blvd. 24	Accounting	027849	3200	Delta	3
4	Mary	F	SW Blvd. 35	IT	027682	3500	Beta	4
4	Mary	F	SW Blvd. 35	IT	027682	3500	Delta	3
5	Donald	M	White House	Marketing	000000	2800	MAGA	5

As you can see, employee Bert is working on two projects. Hence, two tuples are added in the relation to store Bert's involvement into both projects. The other attributes which are unique for this employee are hence duplicated (name, gender, address, department, account, salary). The same holds for Mary.

In this relation, the following rules hold:

1. Every employee has a unique ID. Employees have one name, gender, address, department, account, salary.
2. Every project has exactly one manager.
3. Addresses are not necessarily unique; there may be multiple employees that share the same address. It is, however, forbidden for two employees with the same address to work at the same department (e.g. to avoid nepotism).
4. When a person works on multiple projects, multiple tuples are stored for this person, one for each project and its project manager. The other attributes are duplicated for the employee.
5. The account of a person is unique; every person has one account, and no two employees may have the same account (e.g., due to legal restrictions salaries need to be paid on a personal account).

Question:

- a. (*) For each of the rules 1-5, if possible, state it as a functional dependency or as a multi-valued dependency. If this is not possible, explain why this is not possible.
- b. List all candidate keys of the relation.
- c. (*) Is this relation in Boyce-Codd Normal Form (BCNF)? Explain your answer.
- d. (*) Give a lossless decomposition of the relation in 3NF. List all steps to reach 3NF. Is your decomposition in BCNF?

Problem 4: Concurrency (3 points)

Consider the following sequence of actions, listed in the order they were submitted to the database.

Schedule:

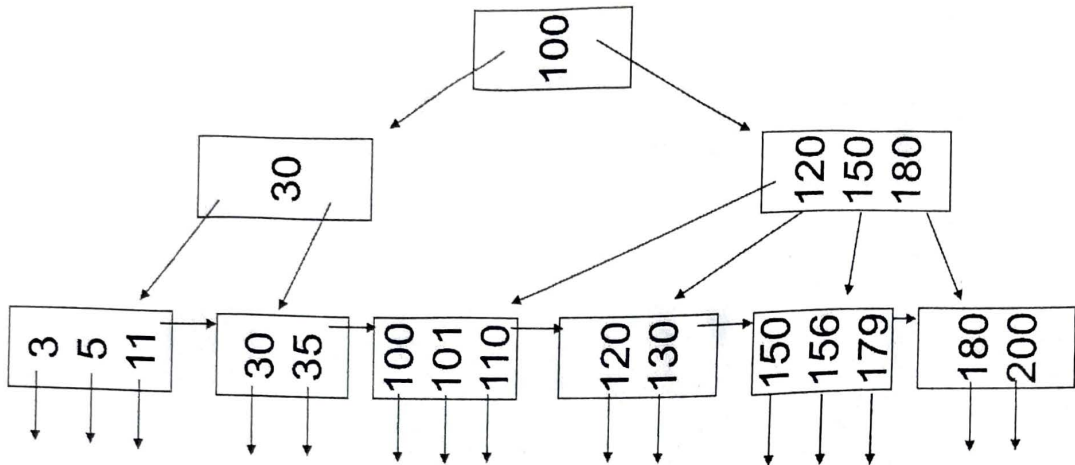
	<i>T1</i>	<i>T2</i>	<i>T3</i>
1.	Read(A)		
2.	Read(B)		
3.		Read(B)	
4.			Read(C)
5.			Write(C)
6.			Read(A)
7.		Write(B)	
8.	Write(A)		
9.		Write(C)	

For the schedule, state which of the following protocols allows it; that is: allows the actions to occur in exactly the order shown above. Explain your answer.

1. (*) Two-phase locking protocol; if you think the schedule is possible, then clearly indicate for each transaction the moment it acquires which type of lock, and when it releases it again.
2. (*) The timestamp-based concurrency protocol with Thomas' rewrite rule. Assume $T1 < T2 < T3$. Show the progression of the read and write timestamps as the transactions progress.
3. (*) The timestamp-based concurrency protocol **without** Thomas' rewrite rule. Assume $T3 < T2 < T1$. Show the progression of the read and write timestamps as the transactions progress.

Problem 5: Indexing (2 points)*

Consider the following B-tree with $n=3$ (maximally 3 keys per node).



Show the updated tree after each of the following operations: insert (a record with key) 125, insert 126, delete 150, delete 120. Your answer should hence consist of 4 B-trees.